

# Computer Arithmetic

# Contents

- **The Arithmetic and Logic Unit**
- **Integer Representation**
  - Sign-Magnitude Representation
  - Twos Complement Representation
  - Converting between Different Bit Lengths
  - Fixed-Point Representation
- **Integer Arithmetic**
  - Negation
  - Addition and Subtraction
  - Multiplication
  - Division
- **Floating-Point Representation**
  - Principles
  - IEEE Standard for Binary Floating-Point Representation
- **Floating-Point Arithmetic**
  - Addition and Subtraction
  - Multiplication and Division
  - Precision Considerations
  - IEEE Standard for Binary Floating-Point Arithmetic

# ALU

- The ALU is that part of the computer that actually performs arithmetic and logical operations on data.
- All of the other elements of the computer system—control unit, registers, memory, I/O—are there mainly to bring data into the ALU for it to process and then to take the results back out.

# Cont.

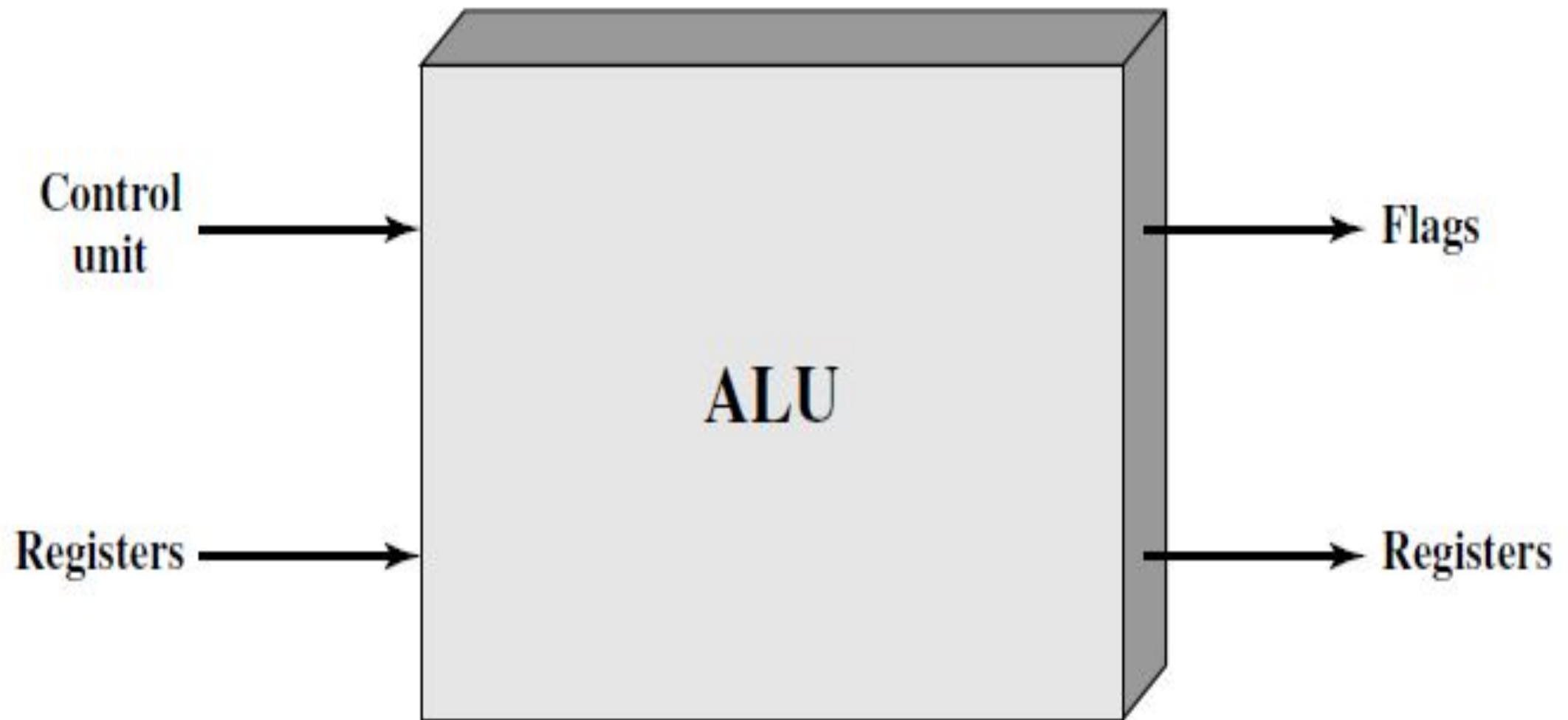


Figure 9.1 ALU Inputs and Outputs

# INTEGER REPRESENTATION

An 8-bit word can represent the numbers from 0 to 255, including

$$00000000 = 0$$

$$00000001 = 1$$

$$00101001 = 41$$

$$10000000 = 128$$

$$11111111 = 255$$

# Sign-Magnitude Representation

- There are several alternative conventions used to represent negative as well as positive integers, all of which involve treating the most significant (leftmost) bit in the word as a sign bit.
- If the sign bit is 0, the number is positive; if the sign bit is 1, the number is negative.

$$\begin{array}{lll} + 18 & = & 00010010 \\ - 18 & = & 10010010 \quad (\text{sign magnitude}) \end{array}$$

$$\begin{array}{lll} +0_{10} & = & 00000000 \\ -0_{10} & = & 10000000 \quad (\text{sign magnitude}) \end{array}$$

# Twos Complement Representation

|                                   |                                                                                                                                                      |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Number of Representations of Zero | One                                                                                                                                                  |
| Negation                          | Take the Boolean complement of each bit of the corresponding positive number, then add 1 to the resulting bit pattern viewed as an unsigned integer. |
| Expansion of Bit Length           | Add additional bit positions to the left and fill in with the value of the original sign bit.                                                        |
| Overflow Rule                     | If two numbers with the same sign (both positive or both negative) are added, then overflow occurs if and only if the result has the opposite sign.  |
| Subtraction Rule                  | To subtract $B$ from $A$ , take the twos complement of $B$ and add it to $A$ .                                                                       |

# 4 bit representation

Table 9.2 Alternative Representations for 4-Bit Integers

| Decimal Representation | Sign-Magnitude Representation | Twos Complement Representation | Biased Representation |
|------------------------|-------------------------------|--------------------------------|-----------------------|
| +8                     | —                             | —                              | 1111                  |
| +7                     | 0111                          | 0111                           | 1110                  |
| +6                     | 0110                          | 0110                           | 1101                  |
| +5                     | 0101                          | 0101                           | 1100                  |
| +4                     | 0100                          | 0100                           | 1011                  |
| +3                     | 0011                          | 0011                           | 1010                  |
| +2                     | 0010                          | 0010                           | 1001                  |
| +1                     | 0001                          | 0001                           | 1000                  |
| +0                     | 0000                          | 0000                           | 0111                  |
| −0                     | 1000                          | —                              | —                     |
| −1                     | 1001                          | 1111                           | 0110                  |
| −2                     | 1010                          | 1110                           | 0101                  |
| −3                     | 1011                          | 1101                           | 0100                  |
| −4                     | 1100                          | 1100                           | 0011                  |
| −5                     | 1101                          | 1011                           | 0010                  |
| −6                     | 1110                          | 1010                           | 0001                  |
| −7                     | 1111                          | 1001                           | 0000                  |
| −8                     | —                             | 1000                           | —                     |

# Converting between Different Bit Lengths

- It is sometimes desirable to take an  $n$ -bit integer and store it in  $m$  bits, where  $n < m$ . In sign-magnitude notation, this is easily accomplished: simply move the sign bit to the new leftmost position and fill in with zeros.

# Cont.

---

|      |    |    |    |   |   |   |   |
|------|----|----|----|---|---|---|---|
| -128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |
|      |    |    |    |   |   |   |   |

(a) An eight-position twos complement value box

|      |    |    |    |   |   |   |   |
|------|----|----|----|---|---|---|---|
| -128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |
| 1    | 0  | 0  | 0  | 0 | 0 | 1 | 1 |

$$\begin{array}{ccccccccccc} & & & & & & +2 & +1 & & \\ -128 & & & & & & & & & = -125 \end{array}$$

(b) Convert binary 10000011 to decimal

|      |    |    |    |   |   |   |   |
|------|----|----|----|---|---|---|---|
| -128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |
| 1    | 0  | 0  | 0  | 1 | 0 | 0 | 0 |

$$\begin{array}{ccccccccccc} & & & & & & +8 & & & \\ -120 = -128 & & & & & & & & & \end{array}$$

(c) Convert decimal -120 to binary

# Cont.

Figure 9.2 Use of a Value Box for Conversion between Twos Complement Binary and Decimal

|     |   |                  |                           |
|-----|---|------------------|---------------------------|
| +18 | = | 00010010         | (sign magnitude, 8 bits)  |
| +18 | = | 0000000000010010 | (sign magnitude, 16 bits) |
| -18 | = | 10010010         | (sign magnitude, 8 bits)  |
| -18 | = | 1000000000010010 | (sign magnitude, 16 bits) |

# Negation

- In sign-magnitude representation, the rule for forming the negation of an integer is simple: invert the sign bit.
- In twos complement notation, the negation of an integer can be formed with the following rules:
  - 1. Take the Boolean complement of each bit of the integer (including the sign bit). That is, set each 1 to 0 and each 0 to 1.
  - 2. Treating the result as an unsigned binary integer, add 1.
- This two-step process is referred to as the **twos complement operation**, or the taking of the twos complement of an integer.

# Cont.

$$\begin{array}{rcl} +18 & = & 00010010 \text{ (twos complement)} \\ \text{bitwise complement} & = & 11101101 \\ & & + \quad \quad \quad 1 \\ & & \hline & & 11101110 = -18 \end{array}$$

As expected, the negative of the negative of that number is itself:

$$\begin{array}{rcl} -18 & = & 11101110 \text{ (twos complement)} \\ \text{bitwise complement} & = & 00010001 \\ & & + \quad \quad \quad 1 \\ & & \hline & & 00010010 = +18 \end{array}$$

# Cont.

- The preceding derivation assumes that we can first treat the bitwise complement of A as an unsigned integer for the purpose of adding 1, and then treat the result as a twos complement integer.
- There are two special cases to consider.
- First, consider  $A = 0$ . In that case, for an 8-bit representation:

$$\begin{array}{rcl} 0 & = & 00000000 \text{ (twos complement)} \\ \text{bitwise complement} & = & 11111111 \\ & + & \underline{1} \\ & & 100000000 = 0 \end{array}$$

# Cont.

- There is *carry* out of the most significant bit position, which is ignored. The result is that the negation of 0 is 0, as it should be.
- The second special case is more of a problem. If we take the negation of the bit pattern of 1 followed by zeros, we get back the same number. For example, for 8-bit words,

$$\begin{array}{rcl} -128 & = & 10000000 \text{ (twos complement)} \\ \text{bitwise complement} & = & 01111111 \\ & & + \quad \quad \quad 1 \\ & & \hline & & 10000000 = -128 \end{array}$$

# Addition and subtraction

|                                                                                                                            |                                                                                                                               |
|----------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| $\begin{array}{r} 1001 = -7 \\ + 0101 = 5 \\ \hline 1110 = -2 \end{array}$ <p>(a) <math>(-7) + (+5)</math></p>             | $\begin{array}{r} 1100 = -4 \\ + 0100 = 4 \\ \hline 10000 = 0 \end{array}$ <p>(b) <math>(-4) + (+4)</math></p>                |
| $\begin{array}{r} 0011 = 3 \\ + 0100 = 4 \\ \hline 0111 = 7 \end{array}$ <p>(c) <math>(+3) + (+4)</math></p>               | $\begin{array}{r} 1100 = -4 \\ + 1111 = -1 \\ \hline 11011 = -5 \end{array}$ <p>(d) <math>(-4) + (-1)</math></p>              |
| $\begin{array}{r} 0101 = 5 \\ + 0100 = 4 \\ \hline 1001 = \text{Overflow} \end{array}$ <p>(e) <math>(+5) + (+4)</math></p> | $\begin{array}{r} 1001 = -7 \\ + 1010 = -6 \\ \hline 10011 = \text{Overflow} \end{array}$ <p>(f) <math>(-7) + (-6)</math></p> |

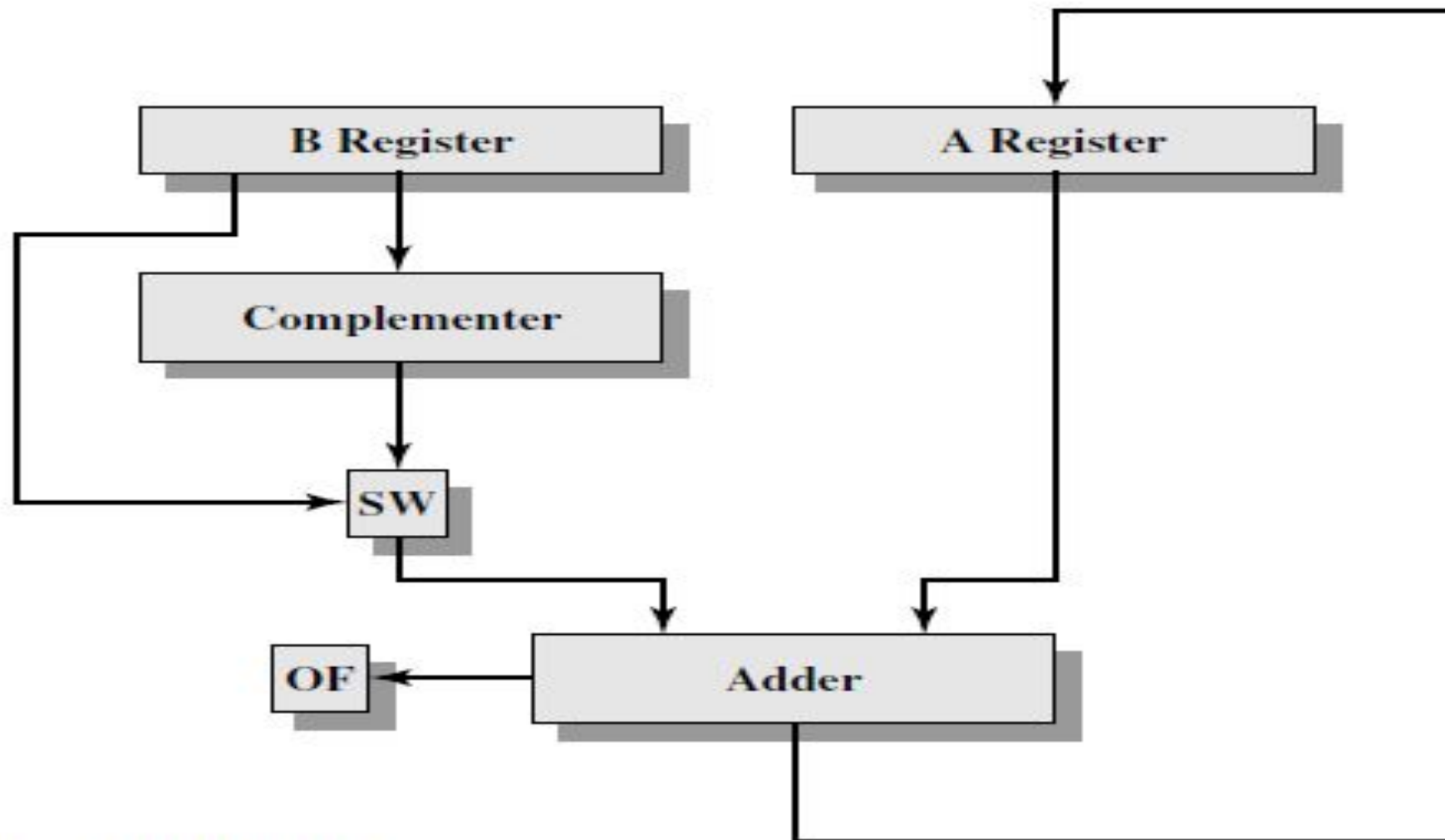
**Figure 9.3** Addition of Numbers in Twos Complement Representation

# Cont.

**SUBTRACTION RULE:** To subtract one number (subtrahend) from another (minuend), take the two's complement (negation) of the subtrahend and add it to the minuend.

|                                                                                                                                                |                                                                                                                                                   |
|------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| $\begin{array}{r} 0010 = 2 \\ + 1001 = -7 \\ \hline 1011 = -5 \end{array}$ <p>(a) M = 2 = 0010<br/>S = 7 = 0111<br/>-S = 1001</p>              | $\begin{array}{r} 0101 = 5 \\ + 1110 = -2 \\ \hline 10011 = 3 \end{array}$ <p>(b) M = 5 = 0101<br/>S = 2 = 0010<br/>-S = 1110</p>                 |
| $\begin{array}{r} 1011 = -5 \\ + 1110 = -2 \\ \hline 11001 = -7 \end{array}$ <p>(c) M = -5 = 1011<br/>S = 2 = 0010<br/>-S = 1110</p>           | $\begin{array}{r} 0101 = 5 \\ + 0010 = 2 \\ \hline 0111 = 7 \end{array}$ <p>(d) M = 5 = 0101<br/>S = -2 = 1110<br/>-S = 0010</p>                  |
| $\begin{array}{r} 0111 = 7 \\ + 0111 = 7 \\ \hline 1110 = \text{Overflow} \end{array}$ <p>(e) M = 7 = 0111<br/>S = -7 = 1001<br/>-S = 0111</p> | $\begin{array}{r} 1010 = -6 \\ + 1100 = -4 \\ \hline 10110 = \text{Overflow} \end{array}$ <p>(f) M = -6 = 1010<br/>S = 4 = 0100<br/>-S = 1100</p> |

# Block Diagram of Hardware for Addition and Subtraction



OF = Overflow bit

SW = Switch (select addition or subtraction)

**Figure 9.6** Block Diagram of Hardware for Addition and Subtraction

# Multiplication

***UNSIGNED INTEGERS*** Figure 9.7 illustrates the multiplication of unsigned binary integers, as might be carried out using paper and pencil. Several important observations can be made:

- 1.** Multiplication involves the generation of partial products, one for each digit in the multiplier. These partial products are then summed to produce the final product.
- 2.** The partial products are easily defined. When the multiplier bit is 0, the partial product is 0. When the multiplier is 1, the partial product is the multiplicand.

Cont.

|          |                   |
|----------|-------------------|
| 1011     | Multiplicand (11) |
| ×1101    | Multiplier (13)   |
| 1011     | Partial products  |
| 0000     |                   |
| 1011     |                   |
| 1011     |                   |
| 10001111 | Product (143)     |

**Figure 9.7** Multiplication of Unsigned Binary Integers

# Number Representation and Arithmetic Operations

- The most natural way to represent a number in a computer system is by a string of bits, called a binary number.
- We will have discussion on binary representation for Integers as its arithmetic operations and then floating-point numbers.

# Integers

- A *bit* is the most basic unit of information in a computer.
  - It is a state of “on” or “off” in a digital circuit.
  - Or “high” or “low” voltage instead of “on” or “off.”
- A *byte* is a group of eight bits.
  - A byte is the smallest possible *addressable* unit of computer storage.
- A *word* is a contiguous group of bytes
  - Word sizes of 16, 32, or 64 bits are most common.
  - Usually a word represents a number or instruction.

# Integers

- Start with the least significant bit (rightmost bit)
- Add each pair of bits
- Include the carry in the addition, if present

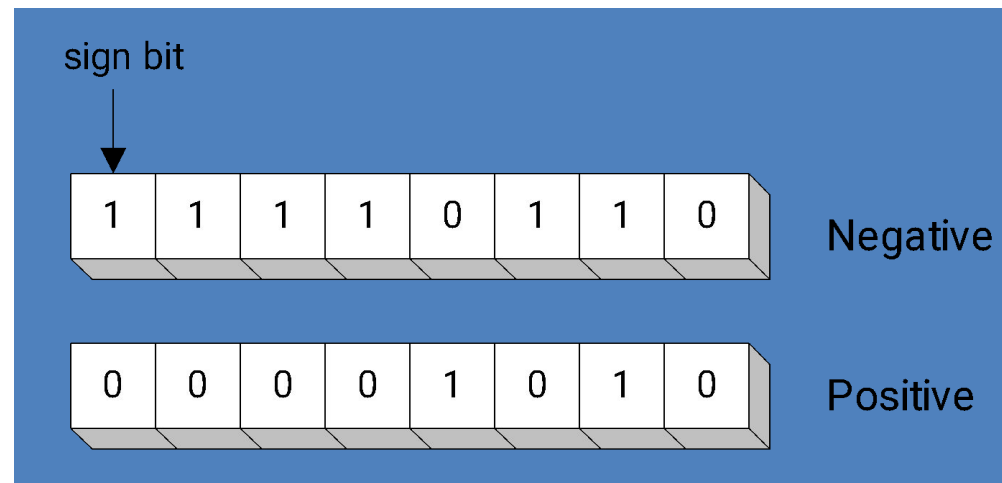
|               |   |   |   |   |   |   |   |   |      |
|---------------|---|---|---|---|---|---|---|---|------|
| carry: 1      |   |   |   |   |   |   |   |   |      |
|               | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | (4)  |
| +             | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | (7)  |
| <hr/>         |   |   |   |   |   |   |   |   |      |
|               | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 1 | (11) |
| bit position: | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |      |

# Integers

- There are three ways in which signed binary numbers may be expressed:
  - Signed magnitude,
  - One's complement and
  - Two's complement.
- In an 8-bit word, signed magnitude representation places the absolute value of the number in the 7 bits to the right of the sign bit.

# Integers

- Highest bit indicates the sign. 1 = negative, 0 = positive



# Integers

- Binary addition is as easy as it gets. You need to know only four rules:

$$\begin{array}{ll} 0 + 0 = 0 & 0 + 1 = 1 \\ 1 + 0 = 1 & 1 + 1 = 10 \end{array}$$

- The simplicity of this system makes it possible for digital circuits to carry out arithmetic operations.
  - We will describe these circuits in Chapter 3.

**Let's see how the addition rules work with signed magnitude numbers . . .**

# Integers

- Example:
  - Using signed magnitude binary arithmetic, find the sum of 75 and 46.

[illegible]

- First, convert 75 and 46 to binary, and arrange as a sum, but separate the (pos

**In this example, we were careful careful to pick two values whose sum would fit into seven bits. If that is not the case, we have a problem.**

# Integers

- Example:
  - Using signed magnitude binary arithmetic, find the sum of 107 and 46.
- We see that the carry from the seventh bit overflows and is discarded, giving us the erroneous result:  
 $107 + 46 = 25.$



Diagram illustrating the binary addition of 107 and 46 using signed magnitude arithmetic. The numbers are represented in 8-bit binary: 107 is 01101011 and 46 is 0101110. The addition is shown with a horizontal line under the second number. The result is 0011001. A carry of 1 is shown above the seventh bit position, with a pink arrow pointing to it from a yellow circle containing the number 1.

$$\begin{array}{r} 0 \quad 1 \quad 1 \quad 0 \quad 1 \quad 0 \quad 1 \quad 1 \\ 0 + 0 \quad 1 \quad 0 \quad 1 \quad 1 \quad 1 \quad 0 \\ \hline 0 \quad 0 \quad 1 \quad 1 \quad 0 \quad 0 \quad 1 \end{array}$$

# Integers

- Signed magnitude representation is easy for people to understand, but it requires complicated computer hardware.
- Another disadvantage of signed magnitude is that it allows two different representations for zero: positive zero and negative zero.
- For these reasons (among others) computer systems employ **complement systems** for numeric value representation.

# Integers

- For example, in 8-bit one's complement;
- + 3 is: 00000011
- - 3 is: 11111100
  - In one's complement, as with signed magnitude, negative values are indicated by a 1 in the high order bit.
- Complement systems are useful because they eliminate the need for special circuitry for subtraction. The difference of two values is found by adding the minuend to the complement of the subtrahend.

# Integers

- With one's complement addition, the carry bit is “carried around” and added to the sum.

– Example: Using one's complement binary arithmetic, find the sum of 48 and -19

We note that 19 in one's complement is 00010011,  
so -19 in one's complement is: 11101100.



The diagram illustrates the process of adding a carry bit in one's complement arithmetic. A yellow circle containing the number '1' is positioned above the first line of the addition. A pink curved arrow originates from this circle and points to the rightmost column of the final sum, indicating that the carry bit is added to the result.

$$\begin{array}{r} \begin{array}{cc} 1 & 1 \end{array} \\ 00110000 \\ 11101100 \\ \hline 00011100 \\ \phantom{00011100} + 1 \\ \hline 00011101 \end{array}$$

# Integers

- Although the “end carry around” adds some complexity, one’s complement is simpler to implement than signed magnitude.
- But it still has the disadvantage of having two different representations for zero: positive zero and negative zero.
- Two’s complement solves this problem.

# Integers

- To express a value *in two's complement*:
  - If the number is positive, just convert it to binary and you're done.
  - If the number is negative, find the one's complement of the number and then add 1.
- Example:
  - In 8-bit one's complement, positive 3 is: 0 0 0 0 0 0 1 1
  - Negative 3 in one's complement is: 1 1 1 1 1 1 0 0
  - Adding 1 gives us -3 in two's complement form: 11111101.

# Integers

|                                          |                |
|------------------------------------------|----------------|
| starting value                           | 00100100 = +36 |
| step1: reverse the bits (1's complement) | 11011011       |
| step 2: add 1 to the value from step 1   | +        1     |
| sum = 2's complement representation      | 11011100 = -36 |

Sum of an integer and its 2's complement must be zero:

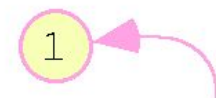
00100100 + 11011100 = 00000000 (8-bit sum)  $\Rightarrow$  Ignore Carry

The easiest way to obtain the 2's complement of a binary number is by starting at the LSB, leaving all the 0s unchanged, look for the first occurrence of a 1. Leave this 1 unchanged and complement all the bits after it.

# Integers

- With two's complement arithmetic, all we do is add our two binary numbers. Just discard any carries emitting from the high order bit.

- Example: Using one's complement binary arithmetic, find the sum of 48 and -19.


$$\begin{array}{r} 1\ 1 \\ 00110000 \\ + 11101101 \\ \hline 00011101 \end{array}$$

We note that 19 in one's complement is: 00010011,  
so -19 in one's complement is: 11101100,  
and -19 in two's complement is: 11101101.

# Integers

- When we use any finite number of bits to represent a number, we always run the risk of the result of our calculations becoming too large to be stored in the computer.
- While we can't always prevent overflow, we can always detect overflow.
- In complement arithmetic, an overflow condition is easy to detect.

# Integers

- Example:
  - Using two's complement binary arithmetic, find the sum of 107 and 46.
- We see that the nonzero carry from the seventh bit **overflows** into the sign bit, giving us the erroneous result:  $107 + 46 = -103$ .

$$\begin{array}{r} \overset{1}{\text{1}} \overset{1}{\text{1}} \overset{1}{\text{1}} \overset{1}{\text{1}} \\ 01101011 \\ + 00101110 \\ \hline 10011001 \end{array}$$

**Rule for detecting two's complement overflow: When the “carry in” and the “carry out” of the sign bit differ, overflow has occurred.**

# Integers

- When subtracting  $A - B$ , convert  $B$  to its 2's complement
- Add  $A$  to  $(-B)$

$$\begin{array}{r}
 00001100\ 00001100 \\
 00000010\ 11111110 \quad (2's\ complement) \\
 -\ 00001010\ 00001010 \quad (same\ result) \\
 \hline
 \end{array}$$



- Carry is ignored, because
  - Negative number is sign-extended with 1's
  - You can imagine infinite 1's to the left of a negative number
  - Adding the carry to the extended 1's produces extended zeros

# Character Representation

- ASCII is the most common encoding schema for characters.
- Alphanumeric characters, operators, punctuation symbols, and control characters are represented by 7-bit codes.
  - A byte is used for the representation and storage.
- The sequential increasing order of codes (alphabets and numeric) when changed to unsigned binary describes sorting operation.
- The low-order four bits of the ASCII describes BCD code.
  - Code for 0 to 9 decimal digits.

Thanks for addition